Zero-Shot Parkinson's Disease Detection from Speech

Models, Datasets, Evaluation, and Colab Acquisition Workflow

Contents

1 Project scope 1

2 Models and input pathways 1

3 Datasets 1

4 Experimental protocol 2

5 Different model pipelines 2
- 5.1 LLaMA 3 8B . . . 2
- 5.2 Audio-language models . . . 2

6 Dataset acquisition and access constraints 2

7 Google Colab project organization 3

8 Combined Colab acquisition and audit code 3

9 What the code does and does not establish 6

10 Readiness checks before inference 6

11 Compute considerations 7

12 Implementation order 7

13 Final note 7

1 Project scope

The project discussed here follows the paper *Zero-Shot Parkinson's Disease Detection from Speech: Comparing Large Audio and Language Models*. The implementation scope is the four LLM/LALM systems discussed in the supplied project notes. A separate second paper/project is to remain outside this implementation phase.

2 Models and input pathways

1. **LLaMA 3 8B (LLM):** receives the paper's 71-dimensional handcrafted acoustic features converted into structured text. It predicts Parkinson's disease (PD) or healthy control.
2. **Qwen2-Audio-7B-Instruct (LALM):** receives the speech waveform with a fixed classification instruction and predicts PD or healthy control.

3. **Pengi (LALM):** processes speech audio using its pretrained audio-language architecture; its pretrained representations are evaluated for the classification task.
4. **Audio-Reasoner (LALM):** receives speech audio and a binary classification instruction, and its zero-shot predictions are evaluated.

LLM here refers to the text-input language-model pathway, whereas LALM refers to a model capable of processing audio. The four systems should not be assumed to receive identical input representations.

3 Datasets

The paper's four datasets, as recorded in the supplied notes, are:

Dataset	Language	Speech type
BenSParX	Bengali	Conversational speech
MDVR-KCL	English	Read speech
IPVS	Italian	Text-dependent utterances
NeuroVoz	Spanish	Spontaneous speech and other speech tasks

The supplied notes report 60 PD and 60 healthy-control participants for BenSParX, and 23 PD and 53 healthy-control participants for NeuroVoz. These cohort counts should be verified against the actual accessible recordings and metadata before experimentation. The availability of raw audio, participant IDs, and diagnosis labels must be checked separately.

4 Experimental protocol

The described phase is zero-shot:

- No task-specific model training or fine-tuning.
- Temperature 0 where supported.
- Random seed 0 where applicable.
- Evaluate balanced accuracy, sensitivity, specificity, ROC-AUC when meaningful scores are available, and Brier score when probabilities are meaningful.
- Use participant-independent evaluation whenever a train/validation/test partition is involved; recordings from the same speaker must not leak across partitions.

The paper-specific evaluation notes further specify subject-level metrics and 95% bootstrap confidence intervals using 10,000 replicates. The notes also state that each recording is evaluated once per model and that segment-level LLaMA predictions are aggregated into subject-level decisions using majority voting. These procedures should be preserved when implementing the paper.

Zero-shot classification does not mean that labelled data cannot be used for evaluation. It means that task-specific training or fine-tuning is not performed. Test labels must not be used to tune prompts or select decision thresholds.

5 Different model pipelines

5.1 LLaMA 3 8B

speech → 71 handcrafted acoustic features → structured text → LLaMA 3 8B → {PD, healthy}.

The supplied notes describe the paper's features as extracted from 10-second speech segments.

5.2 Audio-language models

speech recording → audio preprocessing → Qwen2-Audio / Pengi / Audio-Reasoner → {PD, healthy}.

These models use audio input rather than the LLaMA pathway's structured text representation of handcrafted features.

6 Dataset acquisition and access constraints

The acquisition notes distinguish public downloads, repositories that require inspection, and restricted sources:

- **MDVR-KCL:** the notes identify a Zenodo record and the archive `26_29_09_2017_KCL.zip`, with a direct download URL. This is the dataset for which an automated download was specified.
- **BenSParX:** the notes identify the GitHub repository https://github.com/Riad071/BenSParX. Its listed contents include a CSV, README, license, and script. The repository listing alone does not establish that raw audio recordings are included. Inspect the downloaded files and determine whether the CSV contains features only.
- **IPVS:** the original source is identified as IEEE DataPort. The notes also mention a Hugging Face mirror, https://huggingface.co/datasets/birgermoell/Italian_Parkinsons_Voice_and_Speech, reported at approximately 1.44 GB. A mirror is not automatically equivalent to the original DataPort package; verify participant identifiers, diagnosis labels, speech-task coverage, and usage terms before treating it as the paper's dataset.
- **NeuroVoz:** the notes describe the official Zenodo record as restricted and indicate that access permission or a data-use agreement may be required. Cloning a public code repository does not bypass restrictions on the recordings.

Do not replace the original IPVS corpus with a different Italian corpus. Do not assume that a dataset repository containing acoustic features also contains raw audio. The LALM experiments require accessible audio recordings.

7 Google Colab project organization

The proposed project root is:

```
/content/drive/MyDrive/PD_ZeroShot_Project/
    datasets/
        raw/
            BenSParX/
            MDVR-KCL/
            IPVS/
            NeuroVoz/
        extracted/
            BenSParX/
```

```
            MDVR-KCL/
            IPVS/
            NeuroVoz/
    reports/
```

Keeping raw archives separate from extracted files preserves the original downloads and makes the acquisition process auditable.

8 Combined Colab acquisition and audit code

The following single-cell script reflects the supplied workflow: it mounts Drive, creates folders, downloads the identified MDVR-KCL archive, clones the BenSParX repository for inspection, optionally downloads the cited IPVS Hugging Face mirror, extracts ZIP archives already placed in the raw folders, and creates an inventory and audio manifest. It does not claim to obtain restricted files automatically.

```
from pathlib import Path
import subprocess, zipfile, requests
import pandas as pd
from tqdm.auto import tqdm
from google.colab import drive

drive.mount("/content/drive")

BASE = Path("/content/drive/MyDrive/PD_ZeroShot_Project")
DATA = BASE / "datasets"
RAW = DATA / "raw"
EXTRACTED = DATA / "extracted"
REPORTS = BASE / "reports"
DATASETS = ["BenSParX", "MDVR-KCL", "IPVS", "NeuroVoz"]

for root in [RAW, EXTRACTED, REPORTS]:
    root.mkdir(parents=True, exist_ok=True)
for name in DATASETS:
    (RAW / name).mkdir(parents=True, exist_ok=True)
    (EXTRACTED / name).mkdir(parents=True, exist_ok=True)

# Download MDVR-KCL from the identified Zenodo archive.
mdvr_zip = RAW / "MDVR-KCL" / "26_29_09_2017_KCL.zip"
mdvr_url = (
    "https://zenodo.org/records/2867216/files/"
    "26_29_09_2017_KCL.zip?download=1"
)
if not mdvr_zip.exists() or mdvr_zip.stat().st_size == 0:
    with requests.get(mdvr_url, stream=True, timeout=(30, 180)) as r:
        r.raise_for_status()
        total = int(r.headers.get("content-length", 0))
        with open(mdvr_zip, "wb") as f, tqdm(
            total=total, unit="B", unit_scale=True,
            desc="MDVR-KCL"
        ) as bar:
            for chunk in r.iter_content(1024 * 1024):
                if chunk:
                    f.write(chunk)
                    bar.update(len(chunk))
print("MDVR-KCL archive:", mdvr_zip)
```

```
# Clone BenSParX repository for inspection; this does not
# establish that raw audio is included.
repo = BASE / "BenSParX_repo"
if not repo.exists():
    subprocess.run(
        ["git", "clone", "https://github.com/Riad071/BenSParX.git",
         str(repo)],
        check=True
    )
print("BenSParX repository files:")
for path in sorted(repo.rglob("*")):
    if path.is_file():
        print(path.relative_to(repo))

# Optional community mirror for IPVS. This is not the
# original IEEE DataPort package; verify content and terms.
try:
    from huggingface_hub import snapshot_download
    ipvs_dir = RAW / "IPVS" / "huggingface_mirror"
    ipvs_dir.mkdir(parents=True, exist_ok=True)
    snapshot_download(
        repo_id="birgermoell/Italian_Parkinsons_Voice_and_Speech",
        repo_type="dataset",
        local_dir=str(ipvs_dir)
    )
    print("IPVS mirror downloaded to:", ipvs_dir)
except Exception as exc:
    print("IPVS mirror not downloaded:", repr(exc))
    print("Use the official IEEE DataPort access route if required.")

# Extract ZIP archives found in each raw dataset directory.
extraction_log = []
for name in DATASETS:
    for archive_path in (RAW / name).rglob("*.zip"):
        destination = EXTRACTED / name / archive_path.stem
        marker = destination / ".extraction_complete"
        destination.mkdir(parents=True, exist_ok=True)
        fingerprint = f"{archive_path.stat().st_size}_{archive_path.
            stat().st_mtime_ns}"
        if marker.exists() and marker.read_text().strip() ==
            fingerprint:
            extraction_log.append(
                {"dataset": name, "archive": str(archive_path),
                 "status": "already extracted", "error": ""}
            )
            continue
        try:
            if not zipfile.is_zipfile(archive_path):
                raise ValueError("Not a valid ZIP archive")
            with zipfile.ZipFile(archive_path) as zf:
                bad = zf.testzip()
                if bad:
                    raise ValueError(f"Corrupt member: {bad}")
                zf.extractall(destination)
            marker.write_text(fingerprint)
            extraction_log.append(
                {"dataset": name, "archive": str(archive_path),
                 "status": "extracted", "error": ""}
```

```
            )
        except Exception as exc:
            extraction_log.append(
                {"dataset": name, "archive": str(archive_path),
                 "status": "failed", "error": repr(exc)}
            )

pd.DataFrame(extraction_log).to_csv(
    REPORTS / "extraction_log.csv", index=False
)

# Inventory files and audio recordings.
audio_ext = {".wav", ".mp3", ".flac", ".ogg", ".m4a", ".aac"}
metadata_ext = {".csv", ".tsv", ".xlsx", ".json", ".txt", ".xml"}
inventory_rows, audio_rows = [], []

for name in DATASETS:
    root = EXTRACTED / name
    files = [p for p in root.rglob("*") if p.is_file()
             and p.name != ".extraction_complete"]
    audio_files = [p for p in files if p.suffix.lower() in audio_ext]
    metadata_files = [p for p in files if p.suffix.lower() in
       metadata_ext]

    inventory_rows.append({
        "dataset": name,
        "total_files": len(files),
        "audio_files": len(audio_files),
        "metadata_files": len(metadata_files),
        "total_size_MB": round(
            sum(p.stat().st_size for p in files) / 1e6, 2
        ),
        "status": "Audio␣found" if audio_files else "No␣extracted␣audio
           ␣found"
    })

    for path in audio_files:
        audio_rows.append({
            "dataset": name,
            "file_name": path.name,
            "relative_path": str(path.relative_to(root)),
            "absolute_path": str(path),
            "extension": path.suffix.lower(),
            "size_bytes": path.stat().st_size,
            "participant_id": "",
            "diagnosis": ""
        })

inventory = pd.DataFrame(inventory_rows)
manifest = pd.DataFrame(audio_rows)
inventory.to_csv(REPORTS / "dataset_inventory.csv", index=False)
manifest.to_csv(REPORTS / "audio_manifest.csv", index=False)

print("\\nDATASET␣INVENTORY")
display(inventory)
print("\\nAUDIO␣MANIFEST␣PREVIEW")
display(manifest.head(20))
print("\\nReports␣saved␣in:", REPORTS)
```

9 What the code does and does not establish

The script creates a reproducible folder structure, downloads the identified MDVR-KCL archive, clones the BenSParX repository, attempts the specified IPVS mirror download, extracts available ZIP archives, and creates:

- `extraction_log.csv`: extraction status for discovered ZIP archives.
- `dataset_inventory.csv`: file, audio, metadata, and size counts.
- `audio_manifest.csv`: paths and file properties for each discovered audio recording.

The manifest intentionally leaves `participant_id` and `diagnosis` blank. These must be mapped from verified dataset metadata; they should not be guessed from filenames. The code does not bypass restricted NeuroVoz access, and the BenSParX repository inspection does not prove that raw recordings are present.

10 Readiness checks before inference

Before running model inference, verify:

1. Raw audio recordings are accessible for the intended dataset.
2. Participant-level PD/healthy labels are available.
3. Recording filenames can be mapped reliably to participant IDs.
4. The required speech tasks are present.
5. Resampling and segmentation match the paper (the supplied notes mention 16 kHz and 10-second segments).
6. Dataset licenses and access conditions permit the intended use.

A feature-only BenSParX CSV may support the handcrafted-feature LLaMA pathway if it contains the required features and labels, but it is not sufficient for the three audio-language models that require raw speech. Do not start large-scale inference until the dataset and label mappings have been audited.

11 Compute considerations

The notes recommend Google Colab with a suitable GPU for beginning with Qwen2-Audio-7B-Instruct. A local laptop RTX 3050 may not have enough VRAM for full-precision 7B–8B inference. Quantization may be a practical option, but any deviation from the paper's model precision or inference setup should be recorded.

12 Implementation order

A sensible sequence from the project notes is:

1. Verify dataset access, raw audio, participant IDs, and diagnosis labels.
2. Implement Qwen2-Audio-7B-Instruct using the paper's zero-shot protocol.
3. Implement LLaMA 3 8B using the 71 handcrafted acoustic features.

4. Implement Pengi and Audio-Reasoner after checking their official repositories and inference requirements.
5. Evaluate all four models with the stated subject-level metrics and compare without task-specific fine-tuning.

13 Final note

This document records the project scope and acquisition workflow described in the supplied discussion. Dataset availability, exact file contents, model access, and compatibility must be confirmed in the actual Colab runtime before claiming that the four-dataset experiment is fully reproducible.